

Lab 09 - Storyboard to Video Prompt Pack and Deterministic Animatic

Course: Generative AI for Image and Video Creation (TGS-2020505925) | Version v11.0 | 6 September 2026 Topic 04: Generative AI Video Creation, Animation and Editing Outcome: ELO5 | In-class time: 30 minutes of scheduled practical time

Competency	Statement
Ability A6	Design and apply video analytics algorithms for high-level video analytics tasks
Knowledge K10	Activity tracking, generative models, scene understanding and event discovery

Scenario

A 24-second product film has to be specified before anyone renders anything. Build the shot list, write a continuity bible that pins the things that must not drift between shots, author a structured per-shot video prompt with a per-second audio track, and render a deterministic animatic from your storyboard frames so the timing can be reviewed today.

What you will produce

out/shot-list.csv, out/continuity-bible.md, out/video-prompt-pack.json (three shots with per-second audio), and out/animatic.mp4 - a deterministic OpenCV render of the storyboard frames, explicitly labelled as an animatic and NOT as generative-model output.

Tools: Text editor | Python 3 | opencv-python | OPTIONAL free-tier video model. The lab completes fully offline; no paid service is required.

Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: python3, opencv-python, numpy.

API currency. Everything taught here runs on the base opencv-python wheel. Two things that appear in older tutorials are **not** in that wheel and are deliberately avoided: cv2.TrackerCSRT_create / cv2.TrackerKCF_create and the cv2.quality SSIM module, both of which live in opencv-contrib-python. Where SSIM is needed the course ships its own NumPy implementation.

Workflow

1. Break the brief into a 3-shot list
2. Write the continuity bible

3. Author the structured per-shot prompts
4. Add the per-second audio specification
5. Render and review the deterministic animatic

Files in this folder

Path	What it is
reference/board/shot1_a.png .. shot3_c.png	Nine 1280x720 storyboard frames, three per shot (deterministically rendered with OpenCV - SIMULATED storyboard art, not generative-model output)
data/film-brief.md	The 24-second NorthQuay 'Harbour Line' film brief with the mandatory beats
data/shot-template.json	The per-shot prompt schema, including the audio track structure
data/continuity-fields.json	The continuity fields that must be identical across all shots
animatic.py	Runnable script - see the steps below
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack
out/	Everything you produce (created on first run)

Provenance. Every image and clip in reference/ is either a SIMULATED classroom asset rendered deterministically with OpenCV, an AI-generated reference copied unchanged with a PROVENANCE.md beside it, or a real prerecorded sequence with its source and SHA-256 recorded. Nothing in this folder may be relabelled: a rendered animation is never presented as model output, and a simulated stand-in is never presented as a photograph.

Steps

1. Use the prepared offline OpenCV environment. Work in this lab folder and create out/. The animatic is silent; audio cues are written planning deliverables only. Save out/shot1.json, shot2.json and shot3.json with continuity as an OBJECT, replacing the example placeholder string. Write out/continuity-bible.md, then combine the header and these objects under a shots array in out/video-prompt-pack.json.
2. Read data/film-brief.md. Extract the mandatory beats, the total duration, and the delivery aspect ratios. Note that the brief specifies 24 seconds - for this exercise using a base generation length of 8 seconds per shot, that is three shots, not one clip.
3. Build the shot list as out/shot-list.csv with columns: shot, seconds, purpose, subject_action, camera, location, audio_summary. Three rows of 8 seconds each. Every row must have a PURPOSE - a shot with no job in the narrative gets cut, not shortened.
4. Write the continuity bible. Open data/continuity-fields.json and fill every field. These are the properties that must be byte-identical across all three prompts: product description, wardrobe, model description, time of day, weather, lighting direction, colour palette and lens.
5. State the continuity risk for each field in one clause. For example: 'jacket shell colour - highest risk, because the shell occupies a small frame fraction in shot 3 and the model has least evidence to hold it there.' This is what tells the reviewer where to look.
6. Author shot 1's prompt against data/shot-template.json. The schema separates SUBJECT, ACTION, CAMERA, LOCATION, LIGHTING and NEGATIVE - the same factor separation you compared in Lab 03, extended with the two video-only fields: camera MOVE and shot DURATION.
7. Author shots 2 and 3 by copying shot 1 and changing ONLY the fields the shot list says change. Then diff the three JSON files and confirm that every continuity field is character-identical. Different words can increase drift risk;

identical text does not guarantee visual continuity.

```
python3 -c "import json;a,b,c=[json.load(open(f'out/shot{i}.json')) for i in (1,2,3)];print([k for k in a['continuity'] if not (a['continuity'][k]==b['continuity'][k]==c['continuity'][k])]) or 'CONTINUITY FIELDS IDENTICAL'"
```

8. Add the per-second audio specification to each shot. Selected Veo models can generate audio natively; these per-second cues communicate intent but do not guarantee exact timing. Specify what is heard in each second: ambience, any dialogue with its timing, and any music cue. Write 0-1s, 1-2s ... 7-8s for every shot.
9. Add the negative constraints to every shot. At minimum: no rendered text or logos inside the frame (add those in post so they stay editable and legible), no additional people, no lens flare, and no change to the jacket shell colour.
10. Record the documented generation parameters in the pack header rather than in prose: model id, durationSeconds, aspectRatio and resolution. Verified against the official documentation on 6 September 2026: Veo 3.1 model IDs are veo-3.1-generate-preview, veo-3.1-fast-generate-preview and veo-3.1-lite-generate-preview; durationSeconds accepts 4, 6 or 8; aspectRatio accepts 16:9 (default) or 9:16; resolution accepts 720p, 1080p or 4k with Lite limited to 720p/1080p. 1080p/4k require 8 seconds. Extension is 720p; Lite supports neither referenceImages nor extension. Audio is generated natively. veo-3.0-generate-001 is deprecated.
11. Record the continuity mechanisms the documentation actually offers, so your plan does not depend on hope: on supported non-Lite models, up to three reference images to guide content, image-to-video from a fixed starting frame, and video extension that adds 7 seconds to an existing clip. State which one you would use to hold the product across your three shots, and why.
12. OPTIONAL - if your account has eligible access and quota, submit shot 1 and note that generation is a long-running operation: you submit, then poll until done. Do not block the class on it. If you do not have access, continue; the lab completes without it.

```
# Documented long-running-operation pattern (verified 6 Sep 2026):
from google import genai
from google.genai import types
import time, json
from pathlib import Path

# Before class install google-genai and set GEMINI_API_KEY privately. Access may require
billing.
client=genai.Client()
prompt_text=json.dumps(json.load(open('out/shot1.json')))
operation = client.models.generate_videos(
    model="veo-3.1-generate-preview", prompt=prompt_text,
    config=types.GenerateVideosConfig(duration_seconds=8, aspect_ratio='16:9',
resolution='1080p'))
deadline=time.monotonic()+180
while not operation.done and time.monotonic()<deadline:
    time.sleep(10)
    operation = client.operations.get(operation)
if not operation.done: print('Still pending; record operation.name and continue offline')
elif operation.error: print(operation.error)
elif operation.response and operation.response.generated_videos:
    video=operation.response.generated_videos[0].video
    client.files.download(file=video)
    video.save('out/shot1-generated.mp4')
else: print('No video returned; record status and continue offline')
```

13. Now build the animatic. Load the nine storyboard frames and render them to out/animatic.mp4 at the shot-list timings, with a burned-in shot label, a running timecode and a persistent ANIMATIC banner.

```
python3 animatic.py --board reference/board --shots out/shot-list.csv --out out/animatic.mp4
```

14. Verify the render: open it, confirm the duration matches the shot list total, and confirm the ANIMATIC banner is present on every frame.

```
python3 -c "import cv2;c=cv2.VideoCapture('out/animatic.mp4');n=int(c.get(cv2.CAP_PROP_FRAME_COUNT));f=c.get(cv2.CAP_PROP_FPS);print(n, 'frames', f, 'fps', '%.2fs'%(n/f))"
```

15. Write the provenance line for the animatic and put it in your submission verbatim: 'out/animatic.mp4 is a DETERMINISTIC ANIMATIC rendered from storyboard frames with OpenCV. It is not generative-model output and must not be presented as such.' This matters: mislabelling a deterministic render as model output is a governance failure, not a presentation choice.

16. Review the animatic against the brief's beats. Is 8 seconds enough for shot 2's action? Adjust the action within the fixed 8-second shots, re-render, and record what you changed - that iteration is the entire value of an animatic.

17. Save out/shot-list.csv, out/continuity-bible.md, out/video-prompt-pack.json, out/animatic.mp4 and the provenance line, then complete the evidence checklist.

Verify

The three shot JSON files have character-identical continuity blocks, every shot carries eight per-second audio entries and a negative-constraint list, out/animatic.mp4 plays for the shot-list total duration with a visible ANIMATIC banner, and your submission carries the verbatim provenance line.

Expected outputs

- out/shot-list.csv - 3 rows totalling 24 seconds, each with a stated purpose.
- out/continuity-bible.md - every field from data/continuity-fields.json filled, each with a risk clause.
- out/video-prompt-pack.json - 3 shots, identical continuity blocks, 8 audio entries per shot, negative constraints on every shot.
- A documented-parameters header with model id, durationSeconds, aspectRatio, resolution.
- out/animatic.mp4 - 24 seconds, shot labels, running timecode, persistent ANIMATIC banner.
- The verbatim provenance line for the animatic.

Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

- ☐ out/shot-list.csv with the purpose column filled for all three shots.
- ☐ The continuity diff output showing CONTINUITY FIELDS IDENTICAL.
- ☐ One shot's full per-second audio specification.
- ☐ The negative constraints list used on every shot.
- ☐ The documented parameter header, with the note that veo-3.0-generate-001 is deprecated.
- ☐ Which continuity mechanism you chose (reference images, image-to-video, or extension) and why.
- ☐ out/animatic.mp4 duration check output.
- ☐ The verbatim animatic provenance line.

Troubleshooting

Symptom	What to do
The continuity diff reports differing fields	That is the lab working. Copy the continuity block from shot 1 into shots 2 and 3 programmatically rather than retyping it - retyping is exactly how drift enters.

Symptom	What to do
out/animatic.mp4 is 0 bytes or will not open	The fourcc is unsupported on your build. Try <code>cv2.VideoWriter_fourcc('mp4v')</code> , and confirm every frame you write has the same shape as the writer's declared size - a single mismatched frame silently produces an empty file.
The animatic runs at the wrong speed	The writer FPS and your frame-repeat count must agree: at 24 fps an 8-second shot needs 192 written frames. Compute the repeat count from the shot list, do not hard-code it.
A storyboard frame is a different size from the others	Resize every frame to the writer size on load with <code>cv2.resize</code> . Mixed sizes are the most common cause of a truncated video.
You want to submit the animatic as the finished film	Do not. It is a timing artefact. Its purpose is to let the brief be corrected before any generation budget is spent, and it must stay labelled.
A free-tier video request never completes	Check operation status/error, quota and account access; no free-tier availability is assumed. Use bounded polling, then move on - the deliverable is the prompt pack and the animatic, not the rendered film.

References used in this lab

- [S10] Tschochohei and Schenker (2026), as above - *Ch.7 pp.106-109*.

Spatial versus temporal coherence; early video-GAN failure modes (temporal artifacts/flicker, inconsistent motion, mode collapse) and VGAN/MoCoGAN/TGAN; the 2-D to 3-D U-Net extension for spatiotemporal features; Diffusion Transformers treating video as a sequence of spatiotemporal patches; the 'consistency bottleneck' argument for diffusion over GANs; platform comparison table (core technology, max duration and resolution, primary use case).

- [S11] Tschochohei and Schenker (2026), as above - *Ch.7 pp.110-119*.

Text-to-video, image-to-video and video-with-sound enterprise use cases; long-running operation polling pattern; prompt-driven per-second audio specification; prompt enhancement from a campaign brief.

- [S22] Google AI for Developers - Veo video generation documentation - [*https://ai.google.dev/gemini-api/docs/veo](https://ai.google.dev/gemini-api/docs/veo) (verified 6 Sep 2026)*.

Current Veo model IDs `veo-3.1-generate-preview`, `veo-3.1-fast-generate-preview` and `veo-3.1-lite-generate-preview`, with `veo-3.0-generate-001` deprecated; native audio generation; `durationSeconds` values 4, 6 and 8; `aspectRatio` 16:9 (default) and 9:16; resolution 720p (default), 1080p and 4k with Lite limited to 720p/1080p; up to three reference images to guide content on supported non-Lite models; image-to-video across versions. 1080p/4k require 8-second duration; extension is 720p and adds 7 seconds. Lite does not support `referenceImages` or video extension. The `client.models.generate_videos` long-running-operation polling pattern.

© 2026 Tertiary Infotech Academy Pte Ltd. UEN: 201200696W. Course material for TGS-2020505925, version v11.0.